Spring training day 3:

Topics to be covered:

Spring Test
Spring Unit Integration
Spring Context Caching and Dirties Read annotation for reloading the context
Introduction to JPA
Entity Class and Other Annotations

==========
Unit Testing:
==========

- Done using JUnit framework in Java
- We will be using JUnit 4 with Spring version 4
- Junit will be provided by default in Eclipse
- Right click src folder > New > Source Folder > name it as test > create new Junit class using option given by Eclipse.
- What will be difference between normal Java class & Junit class?
  - In Junit class, there should be at least one method with @Test annotation.
  - A normal Java class can't be run until and unless it has main() method. It can only be compiled. But a Junit class can be run by right click > run unit test.
- Example:

```
public class EmployeeTest {
        @Test
        public void test() {
                assertEquals(101, 45+56);
        }

}
```

- In above unit test, the expected value was 101, and the actual value will be 45+56=101. So the test will pass.

- While writing the test cases for EmployeeBean in Spring, we must load the ApplicationContext first. It will be holding the data.
- Then we will test the data we are getting from the context bean & our actual data.

```
@Test
public void testEmpId() {
        //First we need to load the App Context & get the bean,
        //because it will be holding the information of the bean
        ApplicationContext context = new ClassPathXmlApplicationContext("employee.xml");
        EmployeeBean emp1 = (EmployeeBean) context.getBean("emp");

        //lets test the value of empId
        assertTrue(bean.getEmpId() == 101);
}
```

- The complete test case for EmployeeBean will be:

```
package com.spring.learn;

import static org.junit.Assert.*;

import org.junit.Test;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class EmployeeTest {
```

```
	/*
	 * If we load the ApplicationContext for each test, it will be heavy on memory.
	 * we are using the same bean & loading it again & again.
	 * better we put it out of tests & place it under class instead of test methods.
	 */

	//First we need to load the App Context & get the bean, because it will be holding the
information of the bean
	ApplicationContext context = new ClassPathXmlApplicationContext("employee.xml");
	EmployeeBean bean = (EmployeeBean) context.getBean("emp");

	@Test
	public void testEmpId() {
		//First we need to load the App Context & get the bean, because it will be holding
the information of the bean
		//ApplicationContext context = new
ClassPathXmlApplicationContext("employee.xml");
		//EmployeeBean bean = (EmployeeBean) context.getBean("emp");

		//lets test the value of empId
		assertTrue(bean.getEmpId() == 101);
	}

	@Test
	public void testEmpName() {
		//ApplicationContext context = new
ClassPathXmlApplicationContext("employee.xml");
		//EmployeeBean bean = (EmployeeBean) context.getBean("emp");
		assertEquals("Mark", bean.getEmpName() );
	}

	@Test
	public void testEmpRole() {
		//ApplicationContext context = new
ClassPathXmlApplicationContext("employee.xml");
		//EmployeeBean bean = (EmployeeBean) context.getBean("emp");
		assertEquals("Developer", bean.getRole() );
	}

	@Test
	public void testEmpSalary() {
		//ApplicationContext context = new
ClassPathXmlApplicationContext("employee.xml");
		//EmployeeBean bean = (EmployeeBean) context.getBean("emp");
		assertTrue(450000 == bean.getSalary());
	}

}
```

- Problem with Junit:
  - Till now, we ain't doing any transactions.
  - Our normal Junit doesn't support any transactions like commit or rollback.
  - When we write test cases, we should not write the actual data. It should be taken the rollback operation.
  - So we will use Spring Test module instead of Unit.
  - What will Spring Test module do? - It will initially just load the file.

- **3 Dependencies will be needed for Spring Test:**
  - Javax-inject-1
  - Jsr250-api-1.0
  - Spring-test-5.3.22
- Add them to your project.

==========
Spring Test:
==========
- After adding Spring Test dependency, we can now load the file using ***@ContextConfiguration(location="employee.xml")*** annotation, to tell the location of the config file which is xml.
- Along with this, we will use ***@RunWith(SpringJUnit4ClassRunner.class)*** to integrate our application with the SpringJunitClassRunner.
- Next thing you need to do is injecting the been using @Autowired.
- Suppose we want to reload the context again before any method of after any method is run, we will use **@DirtiesContext** annotation.

```
@DirtiesContext(methodMode=MethodMode.BEFORE_METHOD)
@Test
public void testEmpRole() {
        assertEquals("Developer", bean.getRole() );
}
```

Recap - SpringTest:
- *File will be loaded at the beginning.*
- *We can use @RunWith to integrate with Unit.*
- *@ContextConfiguration - will identify the location to read the file at class level.*
- *@DirtiesContext:*
  - *We have method level*
  - *Before and after method*
  - *We can reload the file depending on requirements*

==========
Now Lets start with JPA:
==========
We will need:
- MySQL - 8.0
  UserName= root
  password- root
  port no- 3306
- In MySQL 8.0 Command Line client, write:
  - show databases;
    - Will display all available database.
  - drop database jpa;
    - Will delete the existing jpa database.
  - create database JPA;
    - Creates new database that is jpa.
  - use jpa;
    - so that database will be use.
  - show tables;
    - will display all available tables inside database.

  create , select, update, delete….

Install MySQL & try above commands.

==========
JDBC
==========
- Helps to achieve data persistence beyond java application.
- Java Database connectivity
- Steps to use JDBC:
    1. Load the driver
    2. Create the connection- using - URL, username, password
    3. Create Query
    4. Execute the query
    5. Close the connection

???WHY ORM???
The problem with JDBC is - for each transaction, all above 5 steps will be performed. Along with that, developer should have knowledge of MySQL, Oracle. While performing the transaction, developer must know the table structure, type of column - string, double, boolean etc, to perform any transaction.

In JDBC, by default everything is auto-commit. If you have hit the command, the operation will be performed. It will not wait for your commit.

To overcome this problem, there is a concept called ORM.
That's where JPA comes into picture.

What was happening in JDBC?
While reading or writing in the database, we are doing it on one to one columns/values.
Why? Because Java is Object-Oriented but database is not. For that, we should take a single value & we should give a single value only.

=> So there is no concept we can send the values for a whole object.
=> ORM allows us to send and receive data in single object to and from Java & DB.

==========
JPA
==========

- Specification of ORM
- We have diff provider of JPA:
    - Eclipse
    - Top Links
    - Hibernate- Yes
    - IBatis
    - myBatis
- JPA belongs to Javax.persistence package
- To use JPA, we must know two Interfaces:
    - EntityManagerFactory
        - Provides connection bw Java & DB with the help of Persistence class & a method createEntityManager() inside it.
        - Inside this method we will provide the JPA, that will be our persistence unit name.
        - In short, here we will go and read the required configuration or xml file where we will be having the driver name, url name, password, either you want to display the query on console or not, either you want to create table or not etc..such info will be available with the help of that it will do the connectivity.
        - Will read the required configuration from persistence.xml file
        - In Spring, we might not be using the EntityManagerFactory, but the DB connectivity will be same. We might use Entity Manager.

In recap:

- *Will read the required configuration from persistence unit name*
- *We need to create one META-INF folder under SRC then create persistence.xml file where will have the required configuration Persistence.createEntityManager("");*

- EntityManager
  - Takes care of what kind of Operation / Transaction are going to happen?
  - insert, delete, update….etc, for all operations, EntityManager will be used to perform the operation.

- By Default JPA will not auto commit the transaction. You need to take care of that part.
- In JDBC, we needed to write a query for each operation. But in JPA, we have some predefined methods:
  - persist()- will do insert operation
  - merge()- will do insert and update
  - find()- will find the single record, based on primary key
  - delete()- deletes one record , based on primary key

- For the above methods, we don't need to write the query.
- However… if you still need to write your own query, we have - JPQL.

- JPQL = Java Persistence Query Language
  - createQuery()
  - createNamedQuery()
  - createNativeQuery()

- Table will be created with the help of Java code & Annotations. Developer don't need to create tables. Below are some useful annotations:

  - @Entity
    - It is mandatory annotation to register a class as entity
    - applied at class level / Java Class
    - used to create table inside database
    - default will take class name as table name

  - @Id
    - It is mandatory.
    - applied variable level / on variable
    - used to make primary key

  - @Table
    - it is optional
    - will apply on class level
    - to change table name inside database

  - @Column
    - it is optional
    - will apply on variable
    - default will take all variables as column in table if we want to change the name of column then will use @Column annotation

  - GenratedValue
    - it is optional, depend on requirement
    - to auto increment the primary key
    - will use only for Integer
    - will increment the value by 1
    - By default will start from 1

Lets jump into the code now.
==========
Add the following jars:

- antlr-2.7.7.jar
- classmate-1.3.0.jar
- dom4j-1.6.1.jar
- hibernate-commons-annotations-5.0.1.Final.jar
- hibernate-core-5.2.8.Final.jar
- hibernate-jpa-2.1-api-1.0.0.Final.jar
- javassist-3.20.0-GA.jar
- jboss-annotations-api_1.2_spec-1.0.0.Final.jar
- jboss-jms-api_2.0_spec-1.0.0.Final.jar
- jboss-logging-3.3.0.Final.jar
- jboss-transaction-api_1.2_spec-1.0.0.Final.jar
- mysql-connector-java-8.0.28

==========
Refer code in GitHub for this section.

JDK & JRE must be 1.8 for the versions compatibility.

- Before running this code, you must create db in your system. Hibernate can create tables not dbs.

To start mysql in terminal, run: mysql -u root -p
mysql> create database JPA;
mysql> show databases;

Mysql logs:

mysql> create database JPA;

```
Query OK, 1 row affected (0.037 sec)

mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| JPA                |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
5 rows in set (0.019 sec)


mysql> use jpa;
Reading table information for completion of table and column
names
```

```
You can turn off this feature to get a quicker startup with
-A

Database changed
mysql> show tables;
+---------------+
| Tables_in_jpa |
+---------------+
| Employee1     |
+---------------+
1 row in set (0.003 sec)

mysql> desc Employee1
    -> ;
+------------+--------------+------+-----+---------+-------+
| Field      | Type         | Null | Key | Default | Extra |
+------------+--------------+------+-----+---------+-------+
| EmployeeId | int          | NO   | PRI | NULL    |       |
| empName    | varchar(255) | YES  |     | NULL    |       |
| role       | varchar(255) | YES  |     | NULL    |       |
| salary     | double       | YES  |     | NULL    |       |
+------------+--------------+------+-----+---------+-------+
4 rows in set (0.019 sec)


mysql> select * from employee1;
+------------+---------+--------+--------+
| EmployeeId | empName | role   | salary |
+------------+---------+--------+--------+
|          1 | Brass   | Player |      0 |
+------------+---------+--------+--------+
1 row in set (0.007 sec)

mysql>
```

- Suppose you already had an empId in db table as 1001 and you again run the code with the same empId. Since, it is the PK, it will throw an error. That's where @GeneratedValue comes in picture.
- We have 4 type of strategy
    1. AUTO- it is default
        - will create one extra table Hibernate-sequence
        - will refer the next value
    2. Identity- will prefer
        - will refer the same table
    ---
    3. Table
    4. Sequence
- When we use GenratedValue value or auto increment, then we should not set id / empId

- Persist()
    - insert operation
    - in persist() - we not suppose to set id because it is auto increment
    - if we so then we will get detached entity pass...

  merge()
    - insert or update
    - when we to merge() then we can set the id
    - If id is new then it will do insert operation with next increment value. It will not follow the value which we provided.
    - If id is existing one, then will do update.

```
public static void mergeOperation() {
        EmployeeEntity entity= new EmployeeEntity();
        entity.setEmpId(101);
        //entity.setEmpId(2);
        entity.setEmpName("Mark");
        entity.setRole("DevOps Developer");
        entity.setSalary(53000.0);
        em.getTransaction().begin();
        em.merge(entity);
        em.getTransaction().commit();
        System.out.println("Employee Addedd!! ");
    }
```

10 mins- as discussed/ explain , please try and let me know

https://www.educaplay.com/learning-resources/27074043-spring_core_fill_in_the_blanks.html